# HW3 - Inverse Kinematics

學號：113950011
姓名：鄭名翔

## 1. Question (5%) — Local 與 Global 座標

Global (world) 座標就是整個場景共用的固定座標系。地板、攝影機、目標球、骨頭的 `end_position`、骨頭的世界旋轉 `bone→rotation`，全都是世界座標。

Local (bone-local) 座標是每根骨頭自己的座標系，原點落在這根骨頭與父骨頭的關節處，三個軸的朝向由 ASF 檔的 `axis` 欄位決定。AMC 檔讀進來、寫在 `posture.bone_rotations[bone→idx]` 的三個 Euler 角度就是 local rotation — 描述「相對父骨頭，這根轉了幾度」。

兩者透過 `forwardSolver` 串接：

```
bone.rotation (world) = parent.rotation × rot_parent_current × Rz·Ry·Rx(local euler)
```

IK 觀測誤差是在 world space ( `target − 末端位置` )，但我能改的只有 local rotation，所以兩種解算法都做了 world → local → clamp DOF/limit → 套回 world 的來回。

## 2. Implementation (10%)

### 2.1 `IKConverged()`

對每條 chain 比末端 ( `jointChain[0]` ) 跟 target 的距離，全部 < `kConvergeEpsilon = 1e-3` 才回 true。`Vector4d` 只取前 3 維算距離。

### 2.2 `clampBoneRotation()`

按 `dofrx/dofry/dofrz` 把沒 DOF 的軸歸零，有 DOF 的軸 clamp 在 `[rxmin, rxmax]` 內。

不過 root 比較特別 — 在 `Skeleton` 建構式裡 rx/ry/rz 都被設為 true，但 ASF 沒給 limits 所以 rxmin/rxmax 預設 0。直接 clamp 會把 root 鎖死。我多加一個 `rxmin ≠ rxmax` 判斷，兩個都 0 就視為沒限制。

### 2.3 `pseudoInverseLinearSolver()`

用 SVD 做 damped least squares：

$x = V \cdot \mathrm{diag}(\sigma_i / (\sigma_i^2 + \lambda^2)) \cdot U^T \cdot b$

$\lambda^2 = 1e\text{-}4$。奇異點附近 $\sigma \approx 0$ 時不會炸成 1/0。Eigen 的 `JacobiSVD<MatrixXd>` 可以直接吃 `Matrix4Xd`。

### 2.4 `JacobianChainSolver()`

每 iter：

1. `delta = target - jointChain[0]`，太小就 break
2. chain 上每根 bone 依 DOF 在 Jacobian 加 column，column = `axis_world × (effector - pivot)`
   - `axis_world` 取該軸在 `(parent.rotation * rot_parent_current).linear()` 下的 UnitX/Y/Z
   - `pivot` 用 `*jointChain[i+1]`（chain-sense 的關節）
3. DLS 求 `dθ`，rad 轉度後加到 posture，clamp
4. Root fix（只 chain 0），FK

### 2.5 `CCDChainSolver()`

每 iter 從 chain 末端 (i=0) 走到 base，每根 bone：

1. `e = cachedEffector - pivot`、`t = target - pivot`
2. `q_world = slerp(I, FromTwoVectors(e, t), step)` — slerp 衰減避免單步轉太猛
3. 投到 local：`q_local_delta = jf_q⁻¹ * q_world * jf_q`，乘 `old_local` 得 `new_local`
4. 轉成 ZYX Euler 寫回 posture，clamp
5. **用 clamp 後的實際旋轉**重算 `cachedEffector` 位置 — PDF p.7 強調的點，我一開始用 clamp 前的，整個就收斂不了

外迴圈結尾做 root fix + FK。

## 3. Discussion (5%)

### 3.1 Step size 的影響

調 `kJacobianIKStepSize` / `kCCDIKStepSize` 試了幾組：

| Step size | Jacobian | CCD |
|---|---|---|
| 0.1 | 很平滑，但要追幾 frame 才到目標 | 末端慢慢逼近，幾乎不抖 |
| 0.3（default） | 跟手又穩，最好用 | 也很穩，偶爾在 limit 邊界抖一下 |
| 0.5 | 奇異點附近開始過衝 | 末端骨轉得太多，後面回拉產生擺動 |
| 1.0 | 直接彈跳 | 第一根骨轉滿後面沒事做，常超過目標再回來 |

整體 0.3 最折衷。

### 3.2 Jacobian vs CCD

| | Jacobian | CCD |
|---|---|---|
| 姿勢 | 全身一起動，比較像人 | 末端骨頭猛轉，base 幾乎不動 |
| 收斂 | 大約 30–80 iter | iter 數較多、但每步算得快 |
| 奇異點 | 沒加 DLS 會炸 | 不會炸但會卡在 limit |
| 跟 DOF/limit 的關係 | clamp 後跟線性解結果不一致，需要多 iter 收斂 | 每根 bone clamp 完馬上更新 cached effector，下一根看到的就是 clamp 後狀態，對 limit 更友善 |

實際拖右手 ball：CCD 會讓 `rfingers / rhand / rwrist` 大幅扭動先把指尖塞過去；Jacobian 是整條手臂從肩膀一起拐過去，比較自然。

### 3.3 FABRIK（Bonus 帶過）

第三種演算法的觀察：position-based 的特性讓它在簡單 chain 上反應比 Jacobian 快，姿勢也比 CCD 自然。但 pure FABRIK 對 DOF/limit 不友善（每根 bone 拿到的目標旋轉被 clamp 後就達不到 FABRIK 理想的 joint 位置），所以我接了一段 CCD 在後面收尾。細節寫在 §4.1。

## 4. Bonus

### 4.1 FABRIK + CCD 混合（第三種 IK）

加了 `IKAlgorithm::FABRIK`、`FABRIKChainSolver()`、UI 下拉選單多一個選項。

**FABRIK 本體**（position-based）：

1. Backward pass：末端 pin 到 target，往 base 走，每段保持原本骨長
2. Forward pass：base pin 回 anchor，往末端走，同樣保持骨長
3. 每個 joint 有目標位置之後，對每根 bone 用 `FromTwoVectors` 算出讓 bone 對齊新方向需要的世界旋轉，再投到 local、clamp

實作踩了三個雷：

- **沒在 bone 之間做 FK**：父 bone 旋轉完，子 bone 的 desiredDir 用舊 pivot 算就跑掉了。改成每根 bone update 完就 forwardSolver，讓下一根用 live pivot 算。
- **跨身體 chain 0（拖手時 chain 跨 root 延伸到對腳當 anchor）**：下半身段 `lhipjoint … lfoot` 的 chain 結構裡 pivot/tip 跟 hierarchy 反向，rotation extraction 公式對它們不適用，直接套會把上半身搞亂。後來偵測 upper section（chain 順序與 hierarchy parent 對齊的前綴）只處理它，下半身交給 root translation fix。
- **嚴格 1e-3 收斂條件下還是常常差一點**：clamp 把每根 bone 的旋轉切掉一些後，末端達不到 FABRIK 理想目標。後面接 `CCDChainSolver` 當 refinement 把殘差收緊。如果 FABRIK 已收斂、CCD 第一輪就 break，幾乎零成本。

### 4.2 Joint Weighting

Jacobian 跟 CCD 都加了「distal 大、base 小」的關節權重：

`w_i = 1 − (1 − w_min) × i / (N−1)`（i=0 distal=1.0、i=N−1 base=`kJointWeightMin=0.35`）

- Jacobian：column 先乘 `w_i` 變 `J' = J·W`，DLS 解 `J' y = b`，再 `dθ = W y`
- CCD：每根 bone 的 slerp step 乘 `w_i`

效果是手指/腳尖反應靈敏，腰跟大腿擺動小，看起來比較像人在動。

### 4.3 UI 顯示 Iter / Residual

`kinematics.h` 加兩個 getter，`inverseIKSolver` 內紀錄 last iter 跟 last max residual，main.cpp 的 Control 面板加一行：

```
ImGui::Text("Iter: %d   Residual: %.5f", kinematics::getLastIterations(), kinematics::getLas
```

debug 用方便很多，調 step size 的時候直接看數字而不用感覺。

### 4.4 DLS Damping

`pseudoInverseLinearSolver` 用 `σ/(σ²+λ²)` 取代 `1/σ`，奇異點不會炸。

### 4.5 CCD Step-size Slerp

`Quaterniond::Identity().slerp(α, q_full)` 衰減 q_world。FABRIK 不用，因為它每根 bone 的目標 direction 互相一致，衰減只會拖慢沒有穩定性收益。

### 4.6 Gimbal-lock Fallback

`matrixToEulerZYX` 在 `cos(θy) ≈ 0` 時改用 `atan2(−R(1,2), R(1,1))` 取 x、z 設 0，避免 NaN。

### 4.7 Root 不被 clamp 鎖死

§2.2 那個 `rxmin ≠ rxmax` 判斷。

## 5. Miscellaneous

### `kinematics.cpp` 以外動到的檔案

| 檔案 | 改了什麼 |
| --- | --- |
| `include/simulation/kinematics.h` | 加 `IKAlgorithm::FABRIK = 2` 跟 `getLastIterations()` / `getLastResidual()` 兩個宣告 |
| `InverseKinematics/main.cpp` | `ikAlgorithms[]` 多 `"FABRIK"`，dispatch 改 switch，Control 面板加一行 ImGui Text |

zip 內附這兩個檔。

## 編譯

VS2022 Community + MSVC 14.38 + CMake + C++17，Windows 11。

## 測試覆蓋

4 個 animation（IK / walk / running / jump）× 4 個 end-effector ball（rfingers / lfingers / rfoot / lfoot）× 3 個 algorithm，都拖過。拖到極遠時 Unstable 燈正常運作、放手會彈回最後穩定位置。

## 已知行為

跨身體 chain 0 在極端拖曳下可看到輕微 anchor drift（量級 < 1e-3），root translation fix 下一 iter 會補回，視覺上幾乎察覺不到。